

AI FanStick - 온디바이스 AI (On-Device LLM) 검토서

작성일: 2026-02-27 목적: 스마트폰 내장 AI(로컬 LLM)를 활용한 AI FanStick 구현 가능성 검토

1. 개요

1.1 온디바이스 AI란?

클라우드 API 대신 스마트폰 내부에서 AI 모델을 직접 실행하는 방식입니다. 네트워크 없이 완전한 오프라인 동작이 가능합니다.

1.2 검토 배경

현재 방식 (Cloud API)	온디바이스 방식 (Local LLM)
Gemini/OpenAI API 호출	스마트폰에서 모델 직접 실행
네트워크 필수	완전 오프라인 가능
API 비용 발생	무료 (일회성 모델 다운로드)
API 키 노출 위험	API 키 불필요
응답 시간 ~2초	응답 시간 ~3-10초 (디바이스에 따라)

2. 온디바이스 LLM 기술 옵션

2.1 주요 프레임워크 비교

프레임워크	개발사	장점	단점	추천도
MediaPipe LLM	Google	공식 지원, 안정적, Gemma 최적화	실험용, 제한된 모델	★★★
llama.cpp	오픈소스	다양한 모델, CPU 최적화	GPU 지원 불완전	★★★
MLC LLM	Apache TVM	GPU 활용, 빠른 속도	설정 복잡, 호환성 이슈	★★
Ollama (Termux)	Ollama Inc.	쉬운 사용	Android 공식 미지원	★

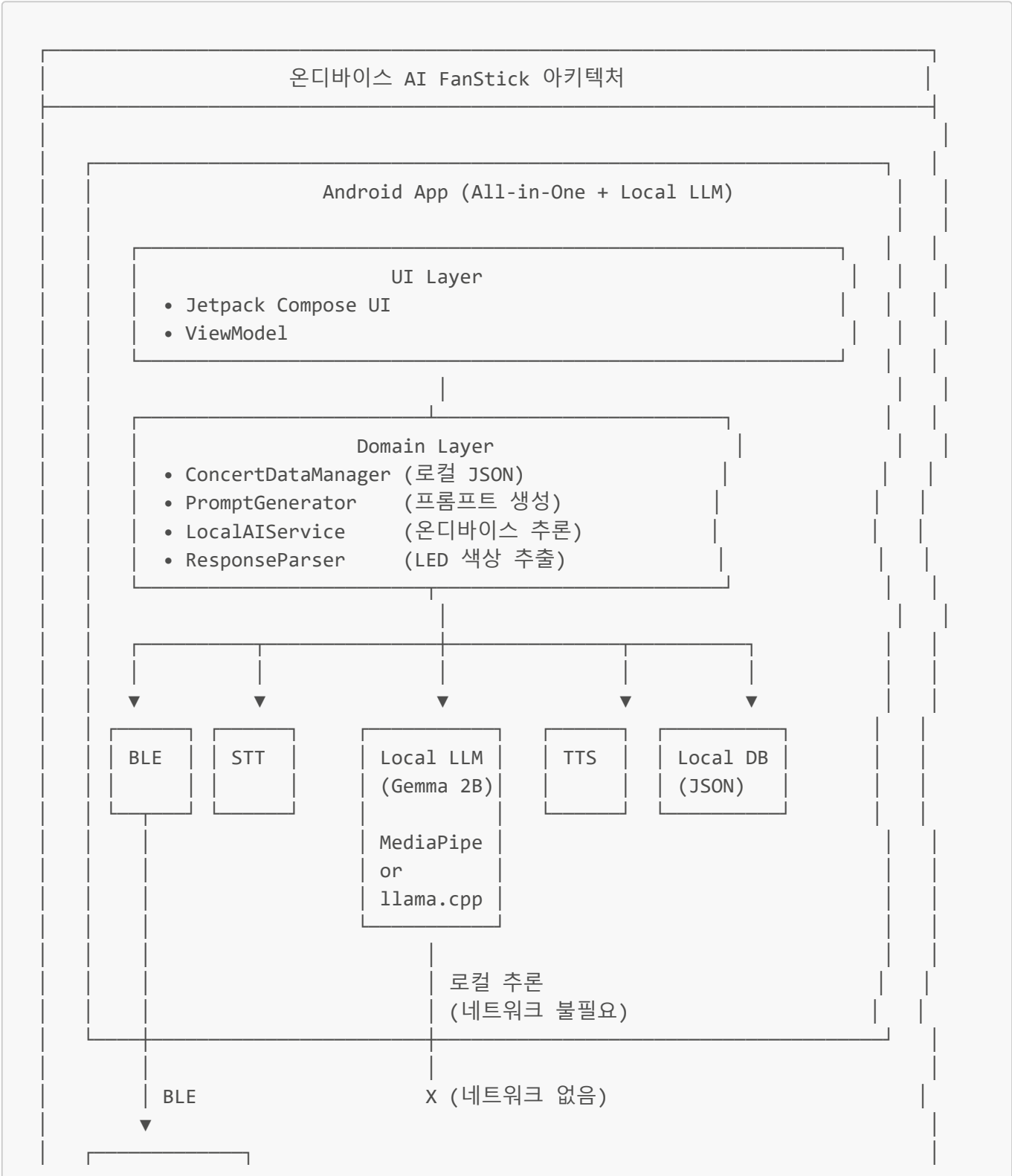
2.2 권장 모델

모델	파라미터	양자화 크기	메모리	용도
Gemma 3n 2B	2B	Q4_K_M: ~1.2GB	~2GB RAM	최적 추천
Gemma 2B	2B	Q4_0: ~1.5GB	~2.5GB RAM	대안

모델	파라미터	양자화 크기	메모리	용도
Gemma 3 4B	4B	Q4_0: ~2.6GB	~4GB RAM	고성능 기기용
Phi-2	2.7B	Q4: ~1.8GB	~3GB RAM	대안
TinyLlama	1.1B	Q4: ~0.7GB	~1.5GB RAM	저사양용

3. 아키텍처 설계

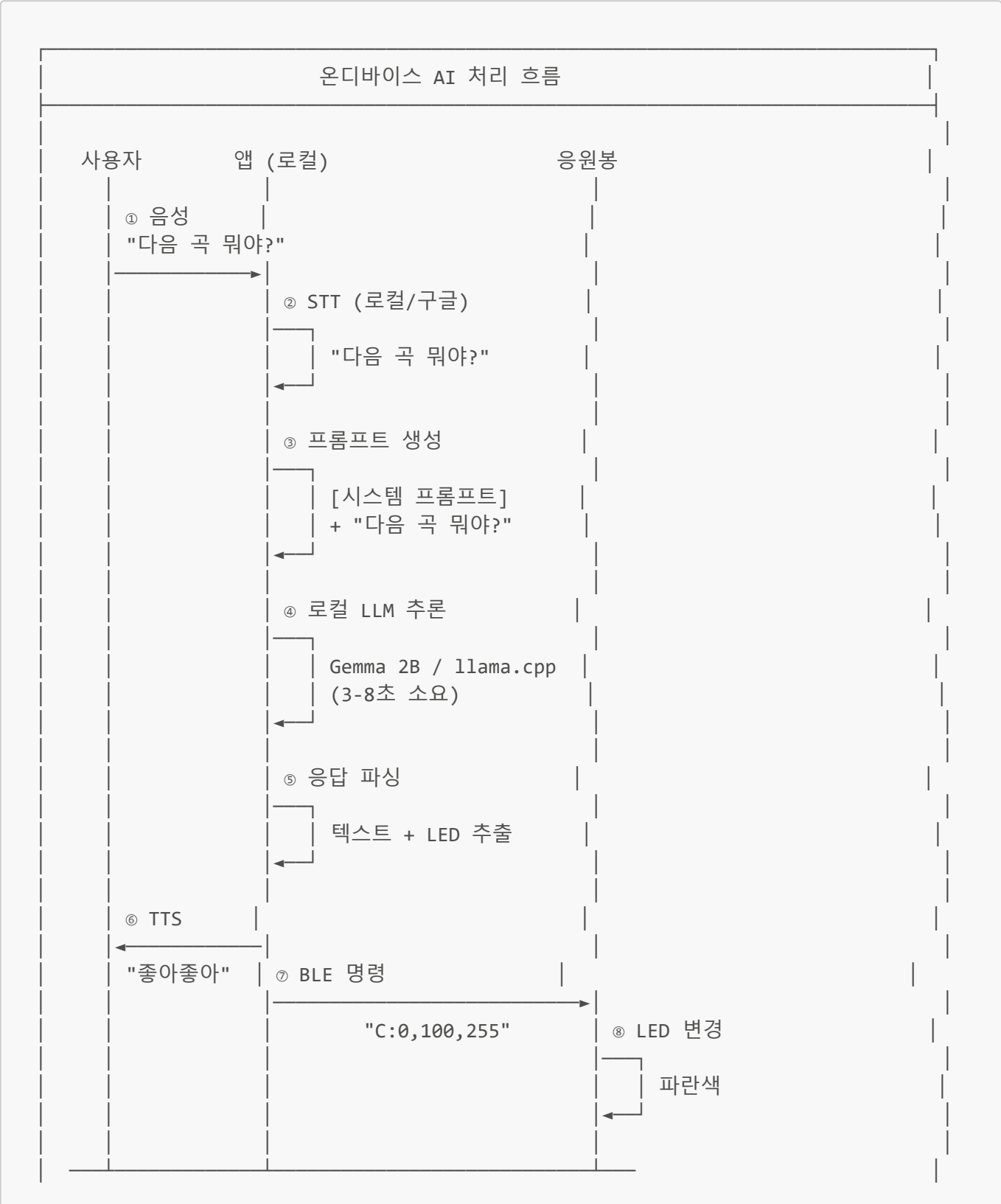
3.1 온디바이스 AI 아키텍처



응원봉
(ESP32)

★ 완전 오프라인 동작 가능! ★

3.2 처리 흐름



【타이밍 비교】

Cloud API 방식: ~2.0초 (네트워크 왕복 포함)
 온디바이스 방식: ~3-8초 (디바이스 성능에 따라)

- * 고성능 기기 (Pixel 8, S24): 3-4초
- * 중간 기기 (Pixel 6, S22): 5-6초
- * 저사양 기기: 8-10초

4. 기술 구현 방안

4.1 MediaPipe LLM Inference (권장)

Google 공식 지원, Gemma 모델 최적화

```
// build.gradle
dependencies {
    implementation("com.google.mediarpipe:tasks-genai:0.10.14")
}

// LocalAIService.kt
class LocalAIService(private val context: Context) {

    private var llmInference: LlmInference? = null

    suspend fun initialize() = withContext(Dispatchers.IO) {
        val options = LlmInference.LlmInferenceOptions.builder()
            .setModelPath("/data/local/tmp/gemma-2b-it-q4_0.bin")
            .setMaxTokens(200)
            .setTemperature(0.8f)
            .setTopK(40)
            .setTopP(0.95f)
            .build()

        llmInference = LlmInference.createFromOptions(context, options)
    }

    suspend fun ask(
        systemPrompt: String,
        userQuestion: String
    ): String = withContext(Dispatchers.IO) {
        val prompt = """$systemPrompt
User: $userQuestion
Assistant: """
```

```

        llmInference?.generateResponse(prompt) ?: "오류가 발생했습니다."
    }

    fun close() {
        llmInference?.close()
    }
}

```

4.2 llama.cpp (대안)

더 다양한 모델 지원, CPU 최적화

```

// llama.cpp JNI 래퍼
class LlamaCppService(private val context: Context) {

    companion object {
        init {
            System.loadLibrary("llama")
        }
    }

    private external fun loadModel(modelPath: String): Long
    private external fun generateText(
        modelHandle: Long,
        prompt: String,
        maxTokens: Int
    ): String
    private external fun freeModel(modelHandle: Long)

    private var modelHandle: Long = 0

    fun initialize(modelPath: String) {
        modelHandle = loadModel(modelPath)
    }

    fun generate(prompt: String, maxTokens: Int = 200): String {
        return generateText(modelHandle, prompt, maxTokens)
    }

    fun close() {
        if (modelHandle != 0L) {
            freeModel(modelHandle)
            modelHandle = 0
        }
    }
}

```

4.3 모델 배포 전략

모델 배포 전략
옵션 1: APK 내장 (비추천) - APK 크기: ~1.5GB (너무 큼) - Play Store 업로드 제한 옵션 2: 앱 최초 실행 시 다운로드 (추천) - APK 크기: ~50MB - 최초 실행 시 모델 다운로드 (~1.2GB) - Hugging Face / Firebase Storage 활용 옵션 3: Android App Bundle + Play Asset Delivery - Google Play의 대용량 에셋 배포 기능 - 필요 시 온디맨드 다운로드 [추천: 옵션 2] 앱 설치 → 첫 실행 → "AI 모델 다운로드 중..." → 완료

5. 장단점 분석

5.1 장점

번호	장점	상세 설명	영향도
1	완전 오프라인	콘서트장 네트워크 불안정해도 동작	★★★
2	API 비용 없음	Gemini/OpenAI 사용료 제로	★★★
3	API 키 불필요	보안 이슈 완전 제거	★★★
4	프라이버시	데이터가 디바이스 외부로 전송 안됨	★★
5	서버 의존성 없음	클라우드 서비스 장애 영향 없음	★★
6	일관된 응답	같은 모델 버전으로 재현 가능한 결과	★

5.2 단점

번호	단점	상세 설명	대응 방안	영향도
1	응답 속도 느림	3-8초 (Cloud: ~2초)	고사양 기기 권장 / 스트리밍 출력	★★★

번호	단점	상세 설명	대응 방안	영향도
2	모델 품질 저하	2B 모델은 GPT-4/Gemini Pro보다 낮음	콘서트 특화 파인튜닝	★★★
3	초기 다운로드	~1.2GB 모델 다운로드 필요	Wi-Fi 환경에서 사전 다운로드	★★
4	저사양 기기 미지원	4GB 이상 RAM 필요	최소 사양 명시	★★
5	발열/배터리	장시간 사용 시 발열, 배터리 소모	사용 빈도 제한 / 쿨다운 안내	★★
6	모델 업데이트	새 모델 배포 시 재다운로드 필요	버전 관리 / 델타 업데이트	★

6. 성능 벤치마크 (예상)

6.1 디바이스별 예상 성능

디바이스	SoC	RAM	예상 속도	지원 여부
Pixel 8 Pro	Tensor G3	12GB	3-4초	☑ 최적
Samsung S24	Snapdragon 8 Gen 3	8GB	3-4초	☑ 최적
Pixel 7	Tensor G2	8GB	4-5초	☑ 양호
Samsung S22	Snapdragon 8 Gen 1	8GB	5-6초	☑ 양호
Pixel 6a	Tensor G1	6GB	6-7초	⚠ 보통
저가형 (4GB RAM)	-	4GB	8-10초	⚠ 느림
구형 (3GB 이하)	-	<4GB	-	✗ 미지원

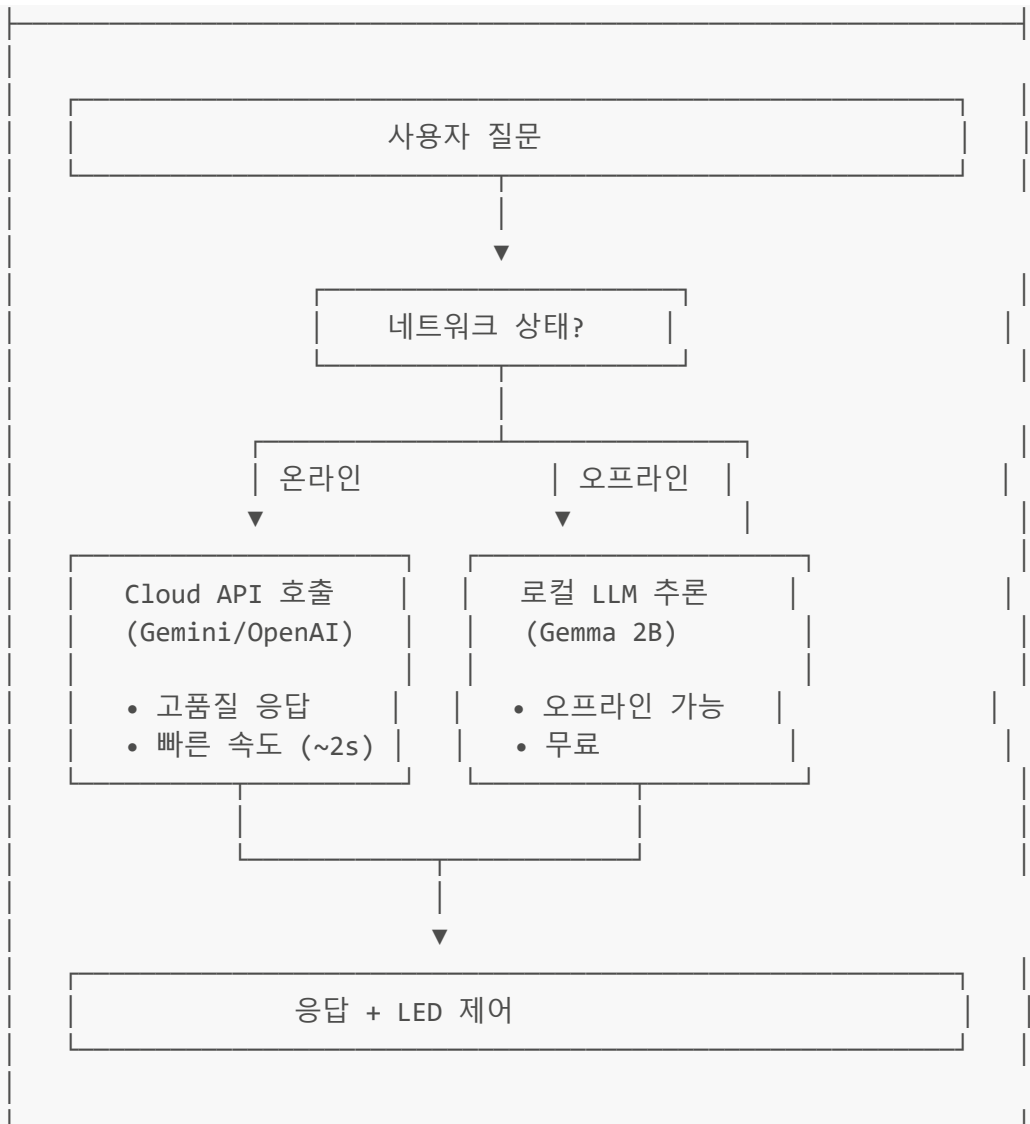
6.2 모델별 예상 성능 (Pixel 8 기준)

모델	크기	품질	속도	추천
Gemma 3n 2B Q4	1.2GB	좋음	3-4초	★★★
TinyLlama 1.1B Q4	0.7GB	보통	2-3초	★★
Gemma 3 4B Q4	2.6GB	매우 좋음	6-8초	★★
Phi-2 Q4	1.8GB	좋음	4-5초	★★

7. 하이브리드 접근법 (권장)

7.1 최적 전략: 온디바이스 + 클라우드 풀백

하이브리드 접근법



7.2 구현 코드

```

class HybridAIService(
    private val context: Context,
    private val concertData: ConcertDataManager,
    private val promptGenerator: PromptGenerator,
    private val cloudApiKey: String = ""
) {
    private var localAI: LocalAIService? = null
    private val cloudAI = AIService(concertData, promptGenerator, cloudApiKey)

    suspend fun initialize() {
        // 백그라운드에서 로컬 모델 로드
        withContext(Dispatchers.IO) {
            try {
                localAI = LocalAIService(context)
                localAI?.initialize()
            } catch (e: Exception) {
                Log.w(TAG, "로컬 AI 초기화 실패, 클라우드만 사용")
            }
        }
    }

```



```

    }
}

suspend fun ask(userQuestion: String): AIResponse {
    // 1. 네트워크 상태 확인
    val isOnline = isNetworkAvailable()

    // 2. 온라인이고 API 키가 있으면 클라우드 우선
    if (isOnline && cloudApiKey.isNotEmpty()) {
        try {
            return cloudAI.ask(userQuestion)
        } catch (e: Exception) {
            Log.w(TAG, "클라우드 API 실패, 로컬 폴백")
        }
    }

    // 3. 오프라인이거나 클라우드 실패 시 로컬 AI
    localAI?.let { local ->
        val systemPrompt = promptGenerator.generate()
        val response = local.ask(systemPrompt, userQuestion)
        val ledColor = parseLedColor(response)
        val cleanResponse = cleanResponse(response)

        return AIResponse(
            success = true,
            response = cleanResponse,
            ledColor = ledColor,
            currentSong = concertData.getCurrentSong()
        )
    }

    // 4. 모두 실패 시 규칙 기반
    return fallbackResponse(userQuestion)
}

private fun isNetworkAvailable(): Boolean {
    val cm = context.getSystemService(Context.CONNECTIVITY_SERVICE) as
ConnectivityManager
    return cm.activeNetwork != null
}
}

```

8. 파인튜닝 가이드 (선택사항)

8.1 콘서트 특화 파인튜닝

온디바이스 모델의 품질을 높이기 위해 콘서트 도메인에 특화된 파인튜닝 가능

[훈련 데이터]

Q: 다음 곡이 뭐야?

A: 다음 곡은 'Butter'야! 노란색 준비해! [LED:255,255,0]

Q: 지금 곡 응원법 알려줘

A: 'Dynamite' 응원법은 "BTS! BTS!"야! [LED:255,215,0]

Q: RM 생일 언제야?

A: RM의 생일은 9월 12일이야! 🎂 [LED:138,43,226]

Q: 보라해가 뭐야?

A: 보라해는 보라색처럼 영원히 사랑한다는 BTS-아미 인사야!
[LED:128,0,128]

... (500-1000개 QA 쌍)

[파인튜닝 도구]

- Unsloth: 4bit 양자화 LoRA 파인튜닝 (가장 효율적)
- PEFT/LoRA: Hugging Face의 표준 도구
- Google Colab: 무료 GPU로 파인튜닝 가능

9. 구현 로드맵

Phase 1: 프로토타입 (2-3일)

- ☐ MediaPipe LLM 라이브러리 통합
- ☐ Gemma 2B 모델 다운로드 기능
- ☐ 기본 추론 테스트

Phase 2: 통합 (2일)

- ☐ PromptGenerator 연동
- ☐ LED 색상 추출 파싱
- ☐ UI 통합

Phase 3: 하이브리드 (1일)

- ☐ 네트워크 상태 감지
- ☐ Cloud/Local 자동 전환
- ☐ 폴백 로직

Phase 4: 최적화 (2일)

- ☐ 모델 로딩 시간 최적화
- ☐ 메모리 관리

- ☐ 배터리 최적화
- ☐ 스트리밍 출력

10. 결론 및 권장 사항

10.1 시나리오별 권장 방식

시나리오	권장 방식	이유
MVP / 데모	Cloud API	빠른 개발, 고품질 응답
상용화 (일반)	하이브리드	안정성 + 오프라인 지원
콘서트장 (오프라인 필수)	온디바이스 우선	네트워크 불안정 대비
저사양 기기 대상	Cloud API only	디바이스 제한

10.2 최종 권장

★★★★ 하이브리드 접근법 권장 ★★★★★

1. 기본: Cloud API (Gemini Flash) 사용

- 빠른 응답 (~2초)
- 고품질 답변

2. 폴백: 온디바이스 (Gemma 2B)

- 오프라인 환경 대응
- API 장애 시 백업

3. 최후: 규칙 기반 응답

- 모든 AI 실패 시
- 기본 콘서트 정보 제공

이 방식으로:

- 평상시: 클라우드의 고품질 응답 활용
- 오프라인: 로컬 AI로 기본 기능 유지
- 최악의 상황: 규칙 기반으로 최소 기능 보장

10.3 디바이스 요구사항

항목	최소 사양	권장 사양
Android	8.0+	12.0+
RAM	4GB	8GB+
저장공간	2GB 여유	4GB 여유

항목	최소 사양	권장 사양
CPU	64-bit ARM	Snapdragon 8 시리즈
대상 기기	Pixel 6+, S21+	Pixel 8+, S23+

11. 참고 자료

프레임워크

- [MediaPipe LLM Inference for Android](#)
- [llama.cpp GitHub](#)
- [MLC LLM](#)
- [Awesome Mobile LLM](#)

모델

- [Gemma 2B GGUF](#)
- [Gemma 3 QAT Models](#)

연구

- [LLM Performance on Mobile Platforms](#)
- [PowerInfer-2: Fast LLM on Smartphone](#)

작성일: 2026-02-27 작성: Claude Code / UTTEC